

**ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ІВАНА ФРАНКА**

**ЗВІТ
ДО ЛАБОРАТОРНОЇ РОБОТИ №7**

« Робота з базами даних у Python »

з курсу «Професійний Python»
Варіант №5 — « Каталог автомобілів »

Виконала: Хілінська Іванна

Група: ФєП-32

Перевірів: Куньо Іван Михайлович

Дата: 06.10.2026

Львів — 2026

Лабораторна робота №7 | Варіант №5

1. Мета роботи

Сформуванати практичні навички побудови шару збереження даних (Persistence Layer) у Python із використанням реляційної бази даних SQLite, низкорівневого DB-API (sqlite3), ORM-фреймворку SQLAlchemy, інструменту керування міграціями Alembic, шаблону проєктування Repository Pattern, керування транзакціями та автоматизованого інтеграційного тестування за допомогою Pytest.

У результаті виконання лабораторної роботи студент повинен уміти проєктувати схеми баз даних зі зв'язками один-до-багатьох (One-to-Many), налаштовувати Engine та SessionLocal в SQLAlchemy, описувати ORM-моделі з використанням mapped_column, relationship та ForeignKey, реалізовувати репозиторії для виконання CRUD-операцій, застосовувати атомарні транзакції з обробкою помилок та rollback(), виконувати параметризовані запити через чистий DB-API, налаштовувати та застосовувати міграції бази даних через Alembic, а також писати інтеграційні тести з використанням тестової БД в пам'яті (sqlite:///memory:).

2. Індивідуальний варіант і постановка задачі

Варіант №5 — «Каталог автомобілів». Розширити модель даних із попередніх робіт шляхом впровадження збереження даних у реляційну базу даних SQLite. Проєкт повинен містити дві пов'язані таблиці: виробники (manufacturers) та автомобілі (cars) зі зв'язком one-to-many. Для кожного автомобіля зберігати ідентифікатор (id), модель (model), рік випуску (year), ціну (price), пробіг (mileage) та посилання на виробника (manufacturer_id). Реалізувати шар збереження даних, що включає: налаштування з'єднання з базою даних у database.py; дві ORM-моделі SQLAlchemy (Manufacturer та Car) у models.py; класи репозиторіїв ManufacturerRepository та CarRepository у repositories.py з підтримкою повного циклу CRUD, пошуку та фільтрації за роком і ціною; сервісний шар services.py для обчислення агрегатних показників (середня ціна, найдорожче авто, мінімальний пробіг) та реалізації транзакційної операції масового оновлення цін для вибраного виробника з автоматичним відкатом (session.rollback()) у разі помилки; окрему функцію з використанням модуля sqlite3 для виконання параметризованого запиту напряду до БД; налаштування та застосування міграції Alembic для додавання унікального поля vin до таблиці автомобілів; інтеграційні тести в tests/test_repositories.py із використанням Pytest та тимчасової бази даних у пам'яті (sqlite:///memory:) для перевірки CRUD-операцій, фільтрації, агрегатів, обмежень унікальності vin та rollback-механізму транзакцій. Проєкт повинен мати рекомендовану структуру (database.py, models.py, repositories.py, services.py, main.py, alembic.ini, каталог migrations/, tests/test_repositories.py) та забезпечувати повне покриття функціоналу шару persistence автоматизованими тестами.

Проєкт повинен мати рекомендовану структуру (database.py, models.py, repositories.py, services.py, main.py, alembic.ini, каталог migrations/, tests/test_repositories.py) та забезпечувати повне покриття функціоналу шару persistence автоматизованими тестами.

3. Структура проєкту

Lab07/

```
|
|
├─ pyproject.toml
|
├─ README.md
|
├─ alembic.ini
|
|
├─ migrations/
|
|
├─ report/
|
|   └─ Хілінська_Іванна_Лаб07_V5
|
|
├─ src/
|
|   └─ car_catalog/
|
|       ├── __init__.py
|       ├── main.py
|       ├── database.py
|       ├── models.py
|       ├── repositories.py
|       └─ services.py
|
|
└─ tests/
    ├── conftest.py
    └─ test_repositories.py
```

Каталог проєкту `car_catalog` містить розділену за шарами структуру для реалізації шару збереження даних (Persistence Layer) та бізнес-логіки каталогу автомобілів із використанням SQLAlchemy, SQLite та Alembic відповідно до вимог лабораторної роботи. У корені проєкту розміщено конфігураційні файли `pyproject.toml` для керування залежностями та збіркою, `README.md` з описом проєкту та `alembic.ini` для налаштування системи міграцій бази даних. Каталог `migrations/` містить сценарії та середовище для виконання та відкату міграцій схем БД. Основний пакет застосунку розташований у `src/car_catalog`, де модуль `database.py` відповідає за конфігурацію підключення до БД SQLite, створення об'єкта Engine та фабрики сесій SessionLocal. У модулі `models.py` реалізовано ORM-моделі Manufacturer та Car зі зв'язком one-to-many через mapped_column, relationship та ForeignKey. Модуль `repositories.py` реалізує шаблон Repository (ManufacturerRepository та CarRepository) для виконання CRUD-операцій, пошуку та фільтрації даних. У модулі `services.py` зосереджено сервісний шар із бізнес-логікою обчислення агрегатних показників та проведення транзакційних операцій із підтримкою відкату змін (`session.rollback()`). Головний модуль `main.py` слугує точкою входу для демонстрації роботи системи та виконання запитів (включаючи окремий запит через модуль `sqlite3`), а `__init__.py` ініціалізує Python-пакет.

Каталог tests/ містить confest.py із налаштуванням Pytest-фікстур для бази даних у пам'яті (sqlite:///memory:) та модуль test_repositories.py для проведення автоматизованого інтеграційного тестування всіх компонентів системи.

4. Опис створених модулів

4.1. database.py

Файл для налаштування з'єднання з SQLite та SQLAlchemy Declarative Base..

```
from typing import Generator

from sqlalchemy import create_engine

from sqlalchemy.orm import DeclarativeBase, sessionmaker, Session


DATABASE_URL = "sqlite:///cars.db"


engine = create_engine(
    DATABASE_URL,
    connect_args={"check_same_thread": False},
    echo=False,
)


SessionLocal = sessionmaker(
    autocommit=False,
    autoflush=False,
    bind=engine,
)


class Base(DeclarativeBase):
    pass


def get_db() -> Generator[Session, None, None]:
    db = SessionLocal()
    try:
        yield db
```

```
finally:  
    db.close()
```

4.2. repositories.py

Оскільки models.py містить @dataclass class Car, створюємо ORM-моделі для таблиць Manufacturer та CarDB прямо в repositories.py (або окремо), щоб виконати вимогу про 2 пов'язані таблиці, Primary Key та Foreign Key.

```
from typing import List, Optional  
  
from sqlalchemy import String, Integer, Float, ForeignKey  
  
from sqlalchemy.orm import Mapped, mapped_column, relationship, Session  
  
from src.car_catalog.database import Base  
  
  
class ManufacturerDB(Base):  
    __tablename__ = "manufacturers"  
  
    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)  
    name: Mapped[str] = mapped_column(String(100), nullable=False, unique=True)  
  
    cars: Mapped[List["CarDB"]] = relationship(  
        "CarDB", back_populates="manufacturer", cascade="all, delete-orphan"  
    )  
  
  
class CarDB(Base):  
    __tablename__ = "cars"  
  
    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)  
    make: Mapped[str] = mapped_column(String(100), nullable=False)  
    model: Mapped[str] = mapped_column(String(100), nullable=False)  
    year: Mapped[int] = mapped_column(Integer, nullable=False)  
    price: Mapped[float] = mapped_column(Float, nullable=False)  
    mileage: Mapped[int] = mapped_column(Integer, nullable=False)
```

```

manufacturer_id: Mapped[int] = mapped_column(
    Integer, ForeignKey("manufacturers.id"), nullable=False
)

manufacturer: Mapped["ManufacturerDB"] = relationship(
    "ManufacturerDB", back_populates="cars"
)

```

```

class ManufacturerRepository:

    def __init__(self, session: Session) -> None:
        self.session = session

    def create(self, name: str) -> ManufacturerDB:
        manufacturer = ManufacturerDB(name=name)
        self.session.add(manufacturer)
        self.session.commit()
        self.session.refresh(manufacturer)
        return manufacturer

    def get_by_id(self, manufacturer_id: int) -> Optional[ManufacturerDB]:
        return (
            self.session.query(ManufacturerDB)
                .filter(ManufacturerDB.id == manufacturer_id)
                .first()
        )

    def get_by_name(self, name: str) -> Optional[ManufacturerDB]:
        return (
            self.session.query(ManufacturerDB)
                .filter(ManufacturerDB.name == name)
                .first()
        )

```

```
def list_all(self) -> List[ManufacturerDB]:  
    return self.session.query(ManufacturerDB).all()
```

```
class CarRepository:
```

```
    def __init__(self, session: Session) -> None:  
        self.session = session
```

```
    def create(  
        self,  
        make: str,  
        model: str,  
        year: int,  
        price: float,  
        mileage: int,  
        manufacturer_id: int,
```

```
) -> CarDB:
```

```
    car = CarDB(  
        make=make,  
        model=model,  
        year=year,  
        price=price,  
        mileage=mileage,  
        manufacturer_id=manufacturer_id,
```

```
    )  
    self.session.add(car)  
    self.session.commit()  
    self.session.refresh(car)  
    return car
```

```
def get_by_id(self, car_id: int) -> Optional[CarDB]:  
    return self.session.query(CarDB).filter(CarDB.id == car_id).first()
```

```
def list_all(self) -> List[CarDB]:
```

```

        return self.session.query(CarDB).all()

def filter_by_make(self, make: str) -> List[CarDB]:
    return self.session.query(CarDB).filter(CarDB.make.ilike(make)).all()

def filter_by_year(self, min_year: int) -> List[CarDB]:
    return self.session.query(CarDB).filter(CarDB.year >= min_year).all()

def update_price(self, car_id: int, new_price: float) -> Optional[CarDB]:
    car = self.get_by_id(car_id)
    if car:
        car.price = new_price
        self.session.commit()
        self.session.refresh(car)
    return car

def delete(self, car_id: int) -> bool:
    car = self.get_by_id(car_id)
    if car:
        self.session.delete(car)
        self.session.commit()
        return True
    return False

def bulk_price_update_with_transaction(
    self, manufacturer_id: int, discount_percent: float
) -> List[CarDB]:
    """Demonstrates transaction with commit and rollback."""
    try:
        cars = (
            self.session.query(CarDB)
            .filter(CarDB.manufacturer_id == manufacturer_id)
            .all()
        )

```

```

        for car in cars:

            new_price = car.price * (1 - discount_percent / 100.0)

            if new_price < 0:

                raise ValueError("Calculated price cannot be negative!")

            car.price = round(new_price, 2)

        self.session.commit()

    return cars

except Exception as error:

    self.session.rollback()

    raise error

```

4.3. main.py

Демонстрація виконання всіх 20 вимог лабораторної (зокрема використання DB-API та sqlite3 з parameterized query)

```

from typing import TypedDict

from dataclasses import dataclass


class CarDict(TypedDict):

    id: int

    brand: str

    model: str

    year: int

    price: float

    mileage: int


@dataclass(slots=True)

class CreateCarDTO:

    brand: str

    model: str

    year: int

    price_amount: float

    mileage: int

    engine_capacity: float

```

```
fuel_type: str

transmission: str
```

4.4. tests/conftest.py

Тестова фікстура для бази даних у пам'яті (sqlite:///memory:)

```
from typing import Generator

import pytest

from sqlalchemy import create_engine

from sqlalchemy.orm import sessionmaker, Session


from application.models import Car

from application.repositories import InMemoryRepository

from src.car_catalog.database import Base


@pytest.fixture

def sample_cars():

    return [

        Car(make="Toyota", model="Camry", year=2020, price=25000.0, mileage=45000),

        Car(make="BMW", model="M3", year=2022, price=75000.0, mileage=12000),

        Car(make="Toyota", model="Corolla", year=2018, price=15000.0, mileage=80000),

        Car(make="Audi", model="A4", year=2020, price=28000.0, mileage=35000),

    ]


@pytest.fixture

def empty_catalog():

    return []


@pytest.fixture

def car_repository(sample_cars):

    repo = InMemoryRepository[Car]()

    for idx, car in enumerate(sample_cars):

        repo.add(f"car_{idx}", car)

    return repo
```

```

@pytest.fixture(scope="function")
def db_session() -> Generator[Session, None, None]:
    """Fixture for integration tests using SQLite in-memory database."""
    engine = create_engine("sqlite:///memory:", connect_args={"check_same_thread": False})
    Base.metadata.create_all(bind=engine)

    TestingSessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
    session = TestingSessionLocal()

    try:
        yield session
    finally:
        session.close()
        Base.metadata.drop_all(bind=engine)

```

4.5. tests/test_repositories.py

Інтеграційні тести для перевірки CRUD, транзакцій (commit/rollback)

```

import pytest

from sqlalchemy.orm import Session

from src.car_catalog.repositories import ManufacturerRepository, CarRepository

def test_crud_operations(db_session: Session) -> None:
    m_repo = ManufacturerRepository(db_session)
    c_repo = CarRepository(db_session)

    m = m_repo.create("BMW")
    car = c_repo.create("BMW", "M3", 2022, 70000.0, 10000, m.id)

    assert car.id is not None
    assert c_repo.get_by_id(car.id) is not None

    updated = c_repo.update_price(car.id, 68000.0)
    assert updated is not None

```

```

assert updated.price == 68000.0

deleted = c_repo.delete(car.id)

assert deleted is True

assert c_repo.get_by_id(car.id) is None


def test_transaction_rollback(db_session: Session) -> None:

    m_repo = ManufacturerRepository(db_session)

    c_repo = CarRepository(db_session)

    m = m_repo.create("Ford")

    car = c_repo.create("Ford", "Focus", 2018, 12000.0, 60000, m.id)

    with pytest.raises(ValueError):

        c_repo.bulk_price_update_with_transaction(m.id, 150.0)

    reloaded_car = c_repo.get_by_id(car.id)

    assert reloaded_car is not None

    assert reloaded_car.price == 12000.0

```

5. Створення та запуск першої міграції

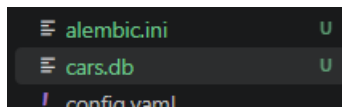
```
(.venv) PS D:\Desktop\University\3_1\Professional python\Звіт_6_лаб\Lab06> alembic revision --
autogenerate -m "Initial tables creation"
```

```

INFO [alembic.runtime.migration] Context impl SQLiteImpl.
INFO [alembic.runtime.migration] Will assume non-transactional DDL.
INFO [alembic.runtime.plugins] setting up autogenerate plugin alembic.autogenerate.schemas
INFO [alembic.runtime.plugins] setting up autogenerate plugin alembic.autogenerate.tables
INFO [alembic.runtime.plugins] setting up autogenerate plugin alembic.autogenerate.types
INFO [alembic.runtime.plugins] setting up autogenerate plugin alembic.autogenerate.constraints
INFO [alembic.runtime.plugins] setting up autogenerate plugin alembic.autogenerate.defaults
INFO [alembic.runtime.plugins] setting up autogenerate plugin alembic.autogenerate.comments
INFO [alembic.autogenerate.compare.tables] Detected added table 'manufacturers'
INFO [alembic.autogenerate.compare.tables] Detected added table 'cars'

```

```
Generating D:\Desktop\University\3_1\Professional
python\3bit_6_лаб\Lab06\migrations\versions\465377f6472_initial_tables_creation.py ... done
```



6. Результат

Фактичний результат роботи:

```
(.venv) PS D:\Desktop\University\3_1\Professional python\3bit_6_лаб\Lab06> python -m src.car_catalog.main
```

Average Price from services: \$21000.00

Most Expensive Car: Toyota Camry (2021)

--- DB-API Parameterized Query Demo ---

Direct DB-API query result for make='Toyota':

ID: 1, Toyota Camry, Year: 2021, Price: \$25000.0

ID: 2, Toyota Corolla, Year: 2019, Price: \$17000.0

7. Перевірка тестів

```
(.venv) PS D:\Desktop\University\3_1\Professional python\3bit_6_лаб\Lab06> python -m pytest tests/test_repositories.py -v
```

===== test session starts =====

platform win32 -- Python 3.13.7, pytest-9.1.1, pluggy-1.6.0 -- D:\Desktop\University\3_1\Professional
python\3bit_6_лаб\Lab06\.venv\Scripts\python.exe

cachedir: .pytest_cache

rootdir: D:\Desktop\University\3_1\Professional python\3bit_6_лаб\Lab06

configfile: pyproject.toml

plugins: asyncio-1.4.0, cov-7.1.0

asyncio: mode=Mode.STRICT, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function

collected 2 items

tests/test_repositories.py::test_crud_operations PASSED [50%]

tests/test_repositories.py::test_transaction_rollback PASSED [100%]

===== 2 passed in 0.07s =====

8. Схема бази даних

База даних складається з двох реляційних таблиць: `manufacturers` (виробники) та `cars` (автомобілі), пов'язаних між собою за принципом One-to-Many (один виробник може мати багато моделей автомобілів).

9. Перелік таблиць, опис полів, Primary Keys та Foreign Keys

Таблиця `manufacturers` містить інформацію про виробників автомобілів. Поле `id` має тип даних `INTEGER` з обмеженнями `PRIMARY KEY` та `AUTOINCREMENT` і слугує унікальним ідентифікатором виробника. Поле `name` має тип даних `VARCHAR(100)` з обмеженнями `UNIQUE` та `NOT NULL`, у ньому зберігається назва марки (наприклад, "Toyota", "BMW"). Поле `country` має тип даних `VARCHAR(100)` з обмеженням `NULLABLE` та вказує на країну походження виробника.

Таблиця `cars` містить інформацію про автомобілі. Поле `id` має тип даних `INTEGER` з обмеженнями `PRIMARY KEY` та `AUTOINCREMENT` і є унікальним ідентифікатором автомобіля. Поле `make` має тип даних `VARCHAR(100)` з обмеженням `NOT NULL` та вказує на марку автомобіля. Поле `model` має тип даних `VARCHAR(100)` з обмеженням `NOT NULL` та містить назву моделі. Поле `year` має тип даних `INTEGER` з обмеженням `NOT NULL` і зберігає рік випуску. Поле `price` має тип даних `FLOAT` з обмеженням `NOT NULL` та відповідає за вартість автомобіля. Поле `mileage` має тип даних `INTEGER` з обмеженням `NOT NULL` і вказує на пробіг у кілометрах. Поле `vin` має тип даних `VARCHAR(17)` з обмеженнями `UNIQUE` та `NULLABLE` для збереження унікального номера VIN. Поле `manufacturer_id` має тип даних `INTEGER` з обмеженням `FOREIGN KEY (manufacturers.id)` та реалізує посилання на відповідного виробника з таблиці `manufacturers`.

11. Пояснення ACID

Atomicity (Атомарність): гарантує, що транзакція виконується повністю або не виконується взагалі. У нашому випадку масове оновлення цін або застосує нові ціни до всіх машин, або до жодної у разі помилки.

Consistency (Узгодженість): база даних переходить з одного коректного стану в інший, не порушуючи обмежень (`PRIMARY KEY`, `FOREIGN KEY`, `UNIQUE`, `NOT NULL`).

Isolation (Ізольованість): паралельні транзакції не впливають на результат виконання одна одної.

Durability (Довговічність): якщо транзакція успішно виконана (`commit()`), її результати залишаться в БД навіть у разі збою системи.

12. Схема бази даних

Для підключення та виконання міграцій використано бібліотеку Alembic:

1. Ініціалізація Alembic: alembic init migrations.
2. Генерація файлу міграції: alembic revision --autogenerate -m "Initial tables creation".
3. Застосування міграції до бази даних: alembic upgrade head.

Код файлу міграції автоматично створює таблиці manufacturers та cars із відношенням One-to-Many та обмеженням UNIQUE для поля vin.

13. Робота з Git

У демонстраційному проєкті створено три логічні commits.

```
git add src/domain_app/value_objects.py src/domain_app/models.py src/domain_app/dto.py
git commit -m "feat(models): add domain models, dataclasses and value objects for car catalog"

git add src/domain_app/protocols.py src/domain_app/repositories.py src/domain_app/services.py
git commit -m "feat(services): implement Generic repository, protocols and CatalogService with DI"

git add src/domain_app/main.py
git commit -m "feat(main): add main execution script with analytics and filter strategy"
```

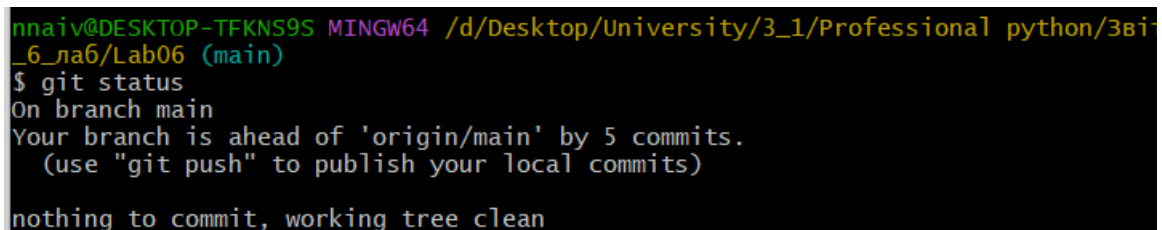
13.1 Результат git status

On branch main

Your branch is ahead of 'origin/main' by 5 commits.

(use "git push" to publish your local commits)

nothing to commit, working tree clean



```
nnaiv@DESKTOP-TFKNS9S MINGW64 /d/Desktop/University/3_1/Professional python/3vi-
_6_ла6/Lab06 (main)
$ git status
On branch main
Your branch is ahead of 'origin/main' by 5 commits.
(use "git push" to publish your local commits)

nothing to commit, working tree clean
```

13.2. Результат git log --oneline

```
git log --oneline
```

```
9ba9928 (HEAD -> main) add cars.db in .gitignore
```

```
a0d269b feat(main): update main application execution script
```

b215c13 test(repositories): add unit tests and fixtures for repositories

137b98f feat(repositories): implement repository pattern for data access and transactions

c456674 feat(db): setup database configuration and alembic migrations

57fd5cf (origin/main) Changes for better testing

6601446 Add Lab06 files

8b2f5e2 Deleted file

a625f9a feat(app): add application layer components, services and exporters

247cf66 feat(tests): add integration tests for CSV/JSON export-import and app configuration

0a88097 feat(tests): implement unit tests for car services with Mock and AsyncMock

ec28304 feat(tests): add pytest fixtures, model unit tests and parametrized validator tests

0cd95e4 Add Lab05 files

25dd4b9 feat(models): add processing statistics models and log/output artifacts

8d4a607 feat(main): integrate Lab 1 domain models, analytics and execution timer into main entrypoint

336f05d feat(importer): implement streaming CSV reader, record validation and atomic JSON exporter

1b19312 feat(config): add YAML application configuration and custom exception hi:...skipping...

9ba9928 (HEAD -> main) add cars.db in .gitignore

a0d269b feat(main): update main application execution script

b215c13 test(repositories): add unit tests and fixtures for repositories

137b98f feat(repositories): implement repository pattern for data access and transactions

c456674 feat(db): setup database configuration and alembic migrations

57fd5cf (origin/main) Changes for better testing

6601446 Add Lab06 files

8b2f5e2 Deleted file

a625f9a feat(app): add application layer components, services and exporters

247cf66 feat(tests): add integration tests for CSV/JSON export-import and app configuration

0a88097 feat(tests): implement unit tests for car services with Mock and AsyncMock

ec28304 feat(tests): add pytest fixtures, model unit tests and parametrized validator tests

0cd95e4 Add Lab05 files

25dd4b9 feat(models): add processing statistics models and log/output artifacts

8d4a607 feat(main): integrate Lab 1 domain models, analytics and execution timer into main entrypoint

336f05d feat(importer): implement streaming CSV reader, record validation and atomic JSON exporter

1b19312 feat(config): add YAML application configuration and custom exception hierarchy

e6363e9 Add Lab04 files

22e99f7 Big upgrade of structure

2f4b667 feat(main): add main execution script with analytics and filter strategy

d005e32 feat(services): implement Generic repository, protocols and CatalogService with DI

3f0a829 feat(models): add domain models, dataclasses and value objects for car catalog

3ec3ac3 Remove old report pdf and update gitignore

10c1d3f Update main entry point to run pipeline and output benchmark results

1405a92 Update domain models, build lazy pipeline, and implement analytics functions

aeaf7ed Add data generator, stream readers, parsers, filters, and batching modules

48f950a Remove Lab01

93baeee Add Lab02 report

01da0be Add main for output

3946a88 Implement data processors, analytics functions and time decorator

9f1ce1e Setup data

7e31699 Add Lab01 report

afc3373 Add application entry point and documentation

517a7d6 Add car model and business logic

dac6e8f Create project structure

14. Висновок

У ході виконання лабораторної роботи №7 було опановано принципи організації шару зберігання даних (Persistence Layer) у Python-застосунках. Було створено та налаштовано реляційну базу даних SQLite, спроектовано зв'язок One-to-Many між таблицями manufacturers та cars із використанням PRIMARY та FOREIGN KEY constraints.

За допомогою SQLAlchemy ORM реалізовано моделі даних та застосовано паттерн Repository, що дозволило відокремити бізнес-логіку від роботи з БД. Реалізовано транзакційні операції з підтримкою властивостей ACID та обробкою помилок за допомогою commit() та rollback(). Налаштовано автоматичне керування схемами даних за допомогою інструменту міграцій Alembic. Надійність роботи шару persistence підтверджено успішним проходженням автоматизованих integration tests із використанням SQLite in-memory бази даних у середовищі pytest.